

Supplementary Material for:

ExecuGraph: Execution-Grounded Multi-Agent Backend Code Synthesis

Implementation Details

This section provides additional implementation details of the ExecuGraph framework.

The system is implemented in Python using the LangGraph workflow orchestration framework and LangChain for LLM interaction. Local model inference is performed using the Ollama platform to ensure reproducibility. The models used include qwen2.5:7b-instruct for planning and review, and qwen2.5-coder:7b-instruct for code generation.

The system follows a graph-based workflow with explicit state propagation across agents, including Planner, Generator, Reviewer, Evaluator, Optimizer, and Explanation agents.

Code Availability

The complete implementation of ExecuGraph, including workflow orchestration, agent definitions, and evaluation scripts, is publicly available at:

<https://github.com/Codeninja026/multi-agent-dsa-backend-logic-synthesis>

The repository includes instructions for setup, execution, and reproduction of experimental results.

Benchmark Problem Set

The evaluation benchmark consists of 30 algorithmic problems spanning three categories:

- Dynamic Programming (e.g., Climbing Stairs, Coin Change, Longest Common Subsequence)
- Graph Algorithms (e.g., Topological Sort, Dijkstra, Cycle Detection)
- Data Structures (e.g., LRU Cache, Min Stack, Linked List operations)

Each problem includes predefined test cases designed to evaluate correctness, edge-case handling, and algorithmic validity.

Reproducibility Instructions

To reproduce the results:

1. Install dependencies listed in the repository.
2. Set up Ollama with the required models:
 - qwen2.5:7b-instruct
 - qwen2.5-coder:7b-instruct

3. Run the main workflow script provided in the repository.
4. Execute benchmark scripts to reproduce evaluation results.

All experiments were conducted using fixed model configurations and deterministic workflow settings.